

useEffect

useEffect is a React Hook that let's perform side effects in functional components.

Side effects are things like:

- Fetching data from APIs
- Setting up subscriptions
- Manually changing the DOM
- Setting timers
- Updating document title

Simple Syntax

```
import {useEffect} from 'react';
```

```
useEffect(() => {
```

```
  // Code to run (side effect)
```

```
  console.log('Effect ran!');
```

```
}, [dependencies]);
```

Pattern 1: Run Once on Mount (when page load)

```
useEffect(() => {
```

```
  console.log('Component mounted');
```

```
  // Runs only once when component first appears
```

```
}, []); // Empty dependency array → means only run when component first show
```

Pattern 2: Run on Every Render

```
useEffect(() => {
```

```
  console.log('Component rendered!');
```

```
  // Runs after every render
```

```
}); // No dependency array
```

Pattern 3: Run When Specific Value Change

```
const [count, setCount] = useState(0);
```

```
useEffect(() => {
```

```
  console.log('count changed to:', count);
```

```
  // Runs when 'count' changes
```

```
}, [count]); // Dependency array with count
```

Pattern 4: Cleanup Function

```
useEffect(() => {
```

```
  count timer = setInterval(() => {
```

```
    console.log('Tick');
```

```
  }, 1000);
```

```
  // Cleanup Function (runs when component unmounts)
```

```
  return () => {
```

```
    clearInterval(timer);
```

```
    console.log('Time cleaned up!');
```

```
  };
```

```
}, []);
```

Fetching Data with useEffect

```
function DataFetcher() {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    fetch('http://jsonplaceholder')  
    .then(response => response.json())  
    .then(data => {  
      setData(data);  
      setLoading(false);  
    })  
    .catch(error => {  
      console.error(error);  
      setLoading(false);  
    });  
  }, []);  
  
  if (!loading)  
    return  
    <div> Data </div>;  
}
```

Component Mount

↓
Effect Runs

↓
Dependency Change?

↓ yes

Effect Cleanup (if exists)

↓

Effect Runs Again

↓

Component Unmounts

↓

Final Cleanup

Lists

The `map()` function transforms each item in an array into JSX elements

```
function FruitList() {  
  const fruits = ['Apple', 'Banana', 'Orange'];  
  return (  
    <ul>  
      {fruits.map(fruit, index) => {  
        <li key={index}>{fruit}</li>  
      }}  
    </ul>  
  );  
}
```

→ React needs a unique key prop for each list item to track changes efficiently

Conditional Rendering

if/else statements

```
function Greeting({isLoggedIn}) {  
  if (isLoggedIn) {  
    return <h1>Welcome back! </h1>  
  } else {  
    return <h1>Please Log In </h1>;  
  }  
}
```

Ternary Statement Operator

```
function Greeting({isLoggedIn}) {  
  return (  
    <div>  
      {isLoggedIn ? (  
        <h1>Welcome back! </h1>  
      ) : (  
        <h1>Please log In </h1>  
      )}  
    </div>  
  );  
}
```

&& Short-Circuit

```
function Notification({newMessage}){  
  return {
```

```
<div>
```

```
<h1> Dashboard </h1>
```

```
{hasNewMessages && <p> You have  
  new messages! </p>
```

```
};  
}
```